



LEADING THE WAY
KHALIFAH • AMĀNAH • IQRA' • RAHMATAN UL-ĀLAMĪN
LEADING THE WORLD



INTERNATIONAL MULTI-AWARD WINNING INSTITUTION FOR SUSTAINABILITY

LAB REPORT 1

MECHATRONICS SYSTEM INTEGRATION

DR. ZULKIFLI BIN ZAINAL ABIDIN

SECTION 1

GROUP 5

MATRIC NUMBER	NAME
2329493	MOHAMED ZAWIR FARIS BIN MOHAMED FAUZI
2319963	MUHAMAD HARITH HAIKAL BIN ZULKIFLI
2316967	MUHAMMAD AIDIL AIMAN BIN WAKIL @ MUHAMMAD SYUKRI

DATE OF SUBMISSION

Wednesday, 18 March 2026

WEEK 2: DIGITAL LOGIC SYSTEM: BASIC LOGIC GATES, ELECTRONIC CIRCUIT

INTERFACING, BASIC ALU, 7-SEGMENT DISPLAY, IC-BASED INTERFACING APPLICATIONS

Abstract

This experiment focuses on understanding digital logic systems and the practical interfacing of a 7-segment display using a Cytron CT-UNO microcontroller. The main objective was to learn how to display numeric sequences using push buttons and to understand how digital logic concepts are applied in hardware. A common cathode 7-segment display was connected to the CT-UNO through digital output pins and programmed to count and reset numbers based on user input. The circuit also included debounce handling to ensure accurate counting. Overall, the experiment successfully demonstrated how digital outputs can be used to control display devices in a simple microcontroller system.

Keywords: Digital Logic, 7-Segment Display, Cytron CT-UNO, Arduino, Push Button, Microcontroller, Debounce

Table of Contents

Abstract	2
1. Introduction.....	3
2. Materials and Equipments.....	2
3. Experimental Setup.....	5
4. Methodology.....	6

5. Data Collection.....	10
6. Data Analysis.....	11
7. Results.....	12
8. Discussion.....	14
9. Conclusion.....	15
10. Recommendations.....	16
11. References.....	17
Appendix A - Circuit Diagram.....	18
Appendix B - Coding Snippets.....	19
Appendix C - 7-Segment Display Results.....	23
Acknowledgments.....	25
Certificate of Originality and Authenticity.....	26

WEEK 2: DIGITAL LOGIC SYSTEM: BASIC LOGIC GATES, ELECTRONIC CIRCUIT INTERFACING, BASIC ALU, 7-SEGMENT DISPLAY, IC-BASED INTERFACING APPLICATIONS

1. Introduction

This week's lab focused on the topic of **Digital Logic Systems**, which are the foundation of all digital electronics such as computers. In digital systems, signals are either ON or OFF, represented by 1 (HIGH) and 0 (LOW). The experiment introduced us to basic logic gates, interfacing techniques, and display devices. Specifically, we used a **7-segment display** to show numbers using digital signals from an Arduino Uno board. The configuration of the 7-segment display pinout is shown in the Figure 1.0 below.

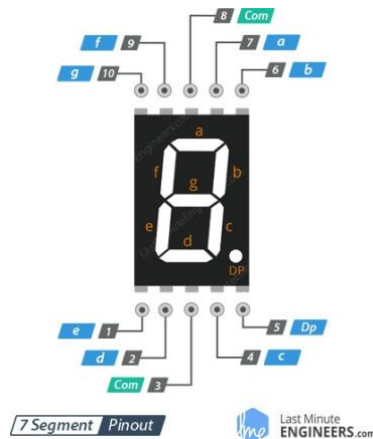


Figure 1.0 Shows the 7-segment display pinout

Our main objectives in this experiment were:

- To learn how a 7-segment display works and how to control each segment.
- To practice connecting electronic components on a breadboard.
- To understand how digital outputs can be programmed to display specific numbers.
- To control the increment sequence of the number display and reset it using push buttons.
- We expected that after connecting all segments properly and uploading the program, the display would show numbers from 0 to 9 correctly and the push button would increase and reset the number.

2.

Materials and Equipments

Cytron CT-UNO	1
Micro USB Cable	1
Common Cathode 7-Segment Display (red)	1
220 Ω Resistors	8
10 k Ω Resistors	2
Pushbuttons	2
Breadboard	1
Jumper Wires	

Experimental Setup

The 7-segment display has 7 LEDs labeled A to G. Each LED lights up when current flows through it. By turning ON specific combinations of these LEDs, we can display numbers from 0 to 9.

The **common anode of the 7-segment display** was connected to 5V (This is the standard for Common Anode). Segment Pins (A to G), connected to D2 to D9 via 220 Ω resistors. Logic: Remains reversed (Active-LOW). To turn a segment ON: The Cytron UNO output must be LOW (0V). This creates a path for the current to flow from the 5V supply, through the LED, and into the pin (sinking current). To turn a segment OFF: The Cytron UNO output must be HIGH (5V). Since both sides of the LED are at 5V, there is no voltage difference, and no current flows.

3.

Two pushbuttons were also connected:

- **Button 1 (Increment):** connected to pin D10.
- **Button 2 (Reset):** connected to pin D11.

Both buttons used **10 k Ω resistors** connected to the GND line to ensure stable logic levels.

Circuit Description:

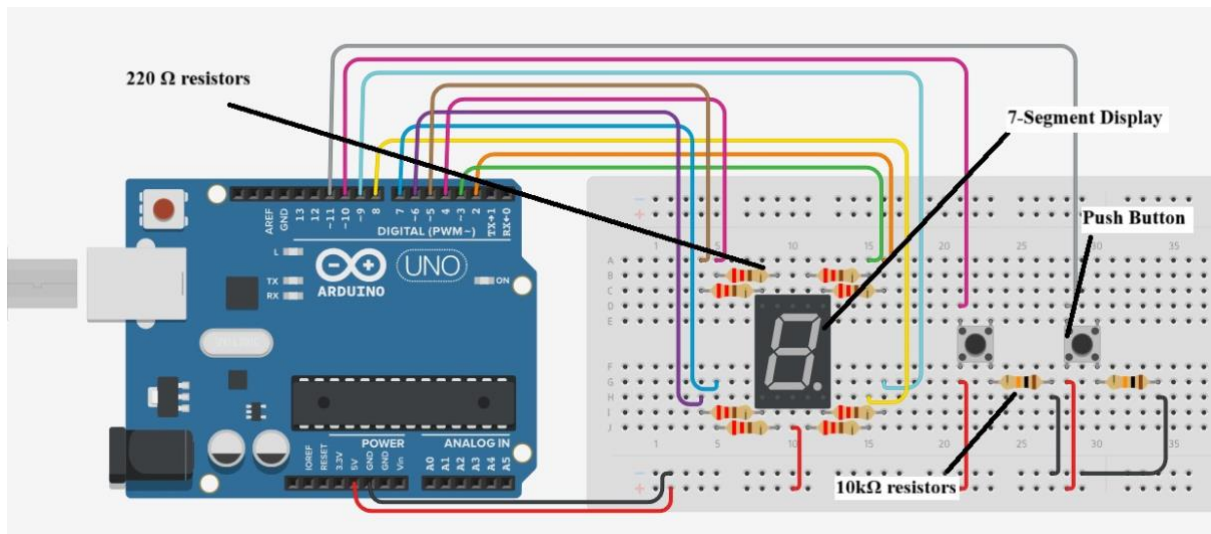


Figure 2.0 Shows the schematic diagram using Tinkercad

- When the Cytron UNO sends a HIGH signal to a pin, that segment lights OFF.
- When the signal is LOW, the segment turns ON.
- The combination of segments turned ON forms a number on the display.

Methodology

4.1. Circuit Assembly

All the connections were made on a breadboard based on Figure 2.0. Each of the seven segment pins (segment a to segment g) was connected through a 220 Ω resistor to Cytron UNO pins D2–D9. The common anode pin was supplied with 5V (logic will be reversed).

4.2. Button Connections

4.

The increment button and reset button were connected to D10 and D11, respectively. Each button was connected to GND through a 10 k Ω resistor on one side and to 5V on the other side.

4.3. Code Upload

The Arduino IDE was used to write and upload the code to Cytron UNO. The program controlled which segments turned ON or OFF to form numbers 0–9.

4.3.1 Example Coding

Coding to display the number from 0 to 9 continuously while following the sequence. Delays were added to make each number stay visible for 1 second before changing.

```
SEVENSEGMENT_MANUAL_PUSHRESET.ino
1 // Define the pins for each segment
2 const int segmentB = 2;
3 const int segmentA = 3;
4 const int segmentF = 4;
5 const int segmentG = 5;
6 const int segmentE = 6;
7 const int segmentD = 7;
8 const int segmentH = 8; //DECIMAL
9 const int segmentC = 9;
10 const int buttonPin = 10; // Pin for the pushbutton
11 const int resetPin = 11; // Pin for the Reset
12 int count = 0;
13
14 void setup()
15 {
16   pinMode(segmentA, OUTPUT);
17   pinMode(segmentB, OUTPUT);
18   pinMode(segmentC, OUTPUT);
19   pinMode(segmentD, OUTPUT);
20   pinMode(segmentE, OUTPUT);
21   pinMode(segmentF, OUTPUT);
22   pinMode(segmentG, OUTPUT);
23   pinMode(segmentH, OUTPUT);
24   pinMode(buttonPin, INPUT_PULLUP); // Button pin 10 to Ground
25   pinMode(resetPin, INPUT_PULLUP); // Reset pin 11 to Ground
26
27   displayNumber(count); // Show 0 at the start
28 }
29
30
31 void loop()
32 {
33   // Check if Reset Button is pressed
34   if (digitalRead(resetPin) == HIGH) {
35     count = 0;
36     displayNumber(count);
37     delay(200);
38   }
39
40   // Check if Increase Button is pressed
41   if (digitalRead(buttonPin) == HIGH)
42   {
43     count++;
44     if (count > 9)
45     {
46       count = 0; // Go back to 0 after 9
47     }
48     displayNumber(count);
49     delay(200); // prevent double-counting
50   }
51 }
```


5.

```

53 void displayNumber(int num)
54 {
55     if (num==0)
56     {
57         // NUMBER 0
58         digitalWrite(segmentA, LOW);
59         digitalWrite(segmentB, LOW);
60         digitalWrite(segmentC, LOW);
61         digitalWrite(segmentD, LOW);
62         digitalWrite(segmentE, LOW);
63         digitalWrite(segmentF, LOW);
64         digitalWrite(segmentG, HIGH); // OFF
65         delay(200);
66     }
67
68     else if (num==1)
69     {
70         // NUMBER 1
71         digitalWrite(segmentA, HIGH);
72         digitalWrite(segmentB, LOW);
73         digitalWrite(segmentC, LOW);
74         digitalWrite(segmentD, HIGH);
75         digitalWrite(segmentE, HIGH);
76         digitalWrite(segmentF, HIGH);
77         digitalWrite(segmentG, HIGH);
78         delay(200);
79     }

```

```

81     else if (num==2)
82     { // NUMBER 2
83         digitalWrite(segmentA, LOW);
84         digitalWrite(segmentB, LOW);
85         digitalWrite(segmentC, HIGH);
86         digitalWrite(segmentD, LOW);
87         digitalWrite(segmentE, LOW);
88         digitalWrite(segmentF, HIGH);
89         digitalWrite(segmentG, LOW);
90         delay(200);
91     }
92
93     else if (num==3)
94     {
95         // NUMBER 3
96         digitalWrite(segmentA, LOW);
97         digitalWrite(segmentB, LOW);
98         digitalWrite(segmentC, LOW);
99         digitalWrite(segmentD, LOW);
100        digitalWrite(segmentE, HIGH);
101        digitalWrite(segmentF, HIGH);
102        digitalWrite(segmentG, LOW);
103        delay(200);
104    }

```

```

106     else if (num==4)
107     {
108         // NUMBER 4
109         digitalWrite(segmentA, HIGH);
110         digitalWrite(segmentB, LOW);
111         digitalWrite(segmentC, LOW);
112         digitalWrite(segmentD, HIGH);
113         digitalWrite(segmentE, HIGH);
114         digitalWrite(segmentF, LOW);
115         digitalWrite(segmentG, LOW);
116         delay(200);
117     }
118
119     else if (num==5)
120     {
121         // NUMBER 5
122         digitalWrite(segmentA, LOW);
123         digitalWrite(segmentB, HIGH);
124         digitalWrite(segmentC, LOW);
125         digitalWrite(segmentD, LOW);
126         digitalWrite(segmentE, HIGH);
127         digitalWrite(segmentF, LOW);
128         digitalWrite(segmentG, LOW);
129         delay(200);
130     }
131

```

```

132     else if (num==6)
133     {
134         // NUMBER 6
135         digitalWrite(segmentA, LOW);
136         digitalWrite(segmentB, HIGH);
137         digitalWrite(segmentC, LOW);
138         digitalWrite(segmentD, LOW);
139         digitalWrite(segmentE, LOW);
140         digitalWrite(segmentF, LOW);
141         digitalWrite(segmentG, LOW);
142         delay(200);
143     }
144
145     else if (num==7)
146     {
147         // NUMBER 7
148         digitalWrite(segmentA, LOW);
149         digitalWrite(segmentB, LOW);
150         digitalWrite(segmentC, LOW);
151         digitalWrite(segmentD, HIGH);
152         digitalWrite(segmentE, HIGH);
153         digitalWrite(segmentF, HIGH);
154         digitalWrite(segmentG, HIGH);
155         delay(200);
156     }

```

6.

```
158 else if (num==8)
159 {
160     // NUMBER 8
161     digitalWrite(segmentA, LOW);
162     digitalWrite(segmentB, LOW);
163     digitalWrite(segmentC, LOW);
164     digitalWrite(segmentD, LOW);
165     digitalWrite(segmentE, LOW);
166     digitalWrite(segmentF, LOW);
167     digitalWrite(segmentG, LOW);
168     delay(200);
169 }
170
171 else if (num==9)
172 {
173     // NUMBER 9
174     digitalWrite(segmentA, LOW);
175     digitalWrite(segmentB, LOW);
176     digitalWrite(segmentC, LOW);
177     digitalWrite(segmentD, LOW);
178     digitalWrite(segmentE, HIGH);
179     digitalWrite(segmentF, LOW);
180     digitalWrite(segmentG, LOW);
181     delay(200);
182 }
183
184 }
```

4.3.2 Task Coding

The coding was written to display a number on the 7-segment display, starting from **0** when the system is powered on. When the “**increase push button**” is pressed, the number on the display increases by **1**. If the number reaches **9**, it will return back to **0** on the next press. When the “**reset push button**” is pressed, the display will immediately reset and show the number **0** again. This operation allows the user to count numbers from 0 to 9 repeatedly using the increase button and reset the count at any time using the reset button.

4.4 Testing and Observation

After uploading, the display was observed to ensure correct numbers appeared in sequence. The increment and reset buttons were pressed to confirm functionality.

5.

Data Collection

During the experiment, the Cytron CT-UNO (Arduino-compatible board) was used to control the circuit and the number displayed on the 7 segment-display. Two main tasks of testing were performed:

5.1. Automatic Display Mode:

- The 7-segment display was programmed to show numbers 1 to 9, followed by 0 in a continuous loop.
- Each number was displayed for approximately one second (1000 ms) before changing to the next number.
- The transition between numbers was smooth, with all relevant segments lighting up correctly for each digit.

5.2. Push Button Controlled Display Mode:

- Two push buttons were connected for increment and reset functions.
- The display started at 0.
- Pressing the increment button increased the number by 1 each time.
- After reaching 9, the next press cycled back to 0.
- The reset button immediately returned the display to 0 regardless of the current number.
- A short delay (200 ms) was used to prevent button bounce from causing double increments.

All results were observed in real time on the breadboard setup and verified through the Serial Monitor for debugging purposes.

Data Analysis

The experiment was designed to verify two types of control logic:

6.

1. Automated sequential display using time delay.
2. Manual control using push buttons and conditional programming logic.

The coding for both setups demonstrated correct digital control principles:

- Digital Write Logic: Each segment (a–g) of the display was controlled individually using `digitalWrite()`.
- Logic Reversal: Because the common cathode was mistakenly connected to 5V, the logic had to be reversed (LOW = ON, HIGH = OFF).
- Debounce Handling: The push button program used a short delay to avoid false triggering from mechanical bounce.
- Modular Programming: Functions `displayNumber()` and `setSegments()` simplified the code structure and made it reusable.

The performance of the circuit was stable after debugging. All numbers from 0–9 were displayed clearly without flickering. The buttons responded accurately with no skipped or repeated counts after debounce implementation.

Results

7.1. Automatic Display Mode:

The 7-segment display successfully showed numbers **0 to 9** in the correct order after supplying the power source to the Cytron UNO. The number is kept looping back to 0 when it reaches 9 until the power source is removed.

7.2. Push Button Controlled Display Mode:

The 7-segment display successfully showed numbers **0 to 9** in the correct order after an increasing push button was pressed and the display **reset to 0** when the reset button was

7.

pressed. The reversed wiring (common cathode to 5V) did not stop the circuit from working, but it **inverted the logic** in coding.

The final result proved that the code logic was correct and the system functioned properly.

Mode	Input Method	Expected Output	Actual Output	Remarks
Automatic	Delay-based loop	$1 \rightarrow 2 \rightarrow \dots \rightarrow 9 \rightarrow 0$	$0 \rightarrow 1 \rightarrow \dots \rightarrow 9 \rightarrow 0$	Works correctly, smooth transition
Manual (Increment)	Push Button 1 (D10)	Increase by +1	Works correctly	Stable with debounce
Manual (Reset)	Push Button 2 (D11)	Reset to 0	Works correctly	Immediate response
Logic Configuration	LOW = ON	Inverted Logic	Corrected in code	Functioned properly

Table 1.0 Shows the result of the lab example and task

Both modes successfully demonstrated digital logic control using Arduino programming. No overheating or circuit failure occurred during the test.

8.

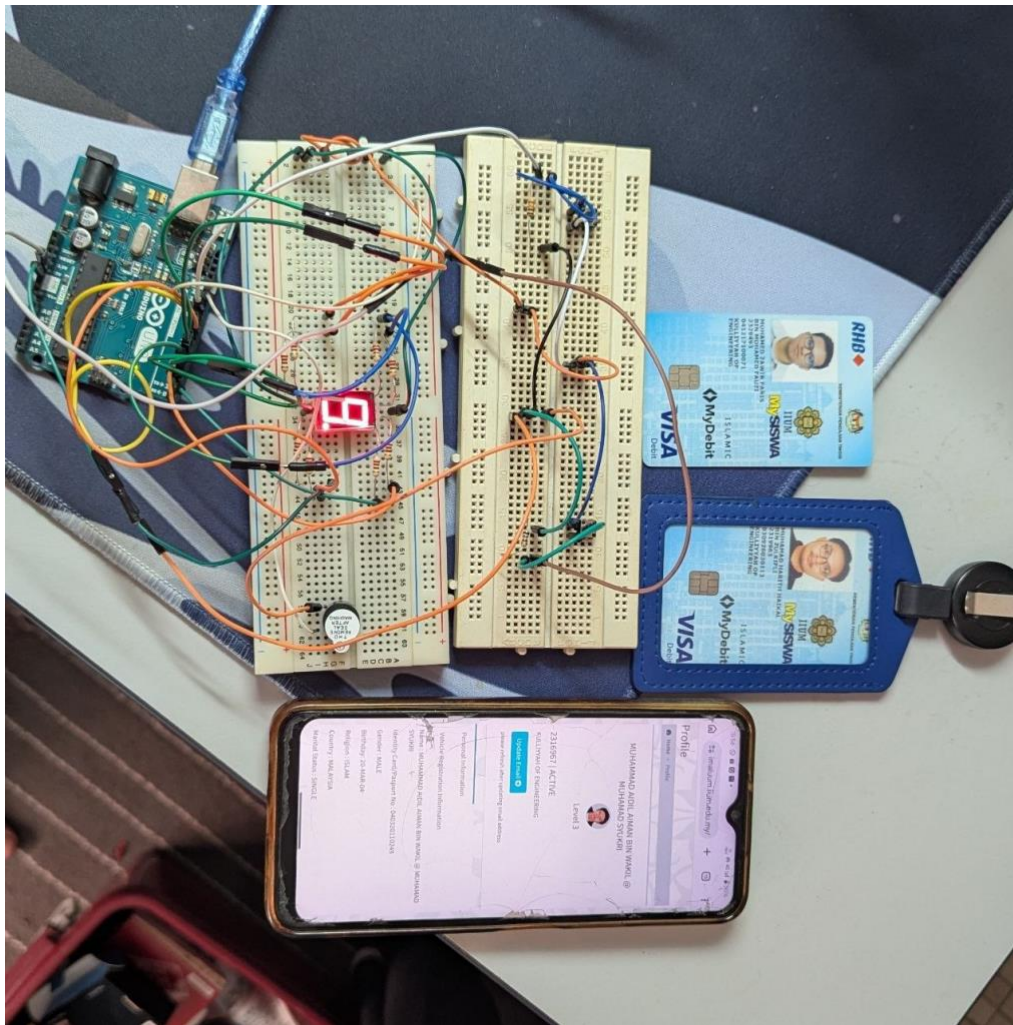


Figure 7.1 Shows the outputs of 1 from 7-segment display

5The Cytron UNO successfully controlled the 7-segment display to show numeric outputs from 0 to 9, as demonstrated in Figure 4.1.

Additional images of the display showing all digits (0–9) are provided in *Appendix C* for reference.

8. Discussion

This experiment showed how a Cytron CT-UNO can be used to control a 7-segment display with the help of push buttons and digital output pins. The main idea was to make the display show numbers that increase when one button is pressed and reset when another button is pressed.

In this experiment, the 7-segment display was accidentally connected in the wrong way which is why the common cathode pin was connected to 5V instead of GND. Normally, for a common cathode display, the common pin should be connected to ground. Because of this mistake, the logic became opposite. The display turned ON when the signal was LOW instead of HIGH. This issue was fixed through coding by reversing the logic in the program, showing how flexible microcontrollers can be.

This experiment also proves that coding can sometimes fix minor hardware wiring mistakes. For future experiments, it is recommended to connect the common cathode pin to GND to follow the standard setup and make troubleshooting easier.

Possible sources of error include button debounce, which can cause the number to increase more than once with a single press if there is no proper delay in the code. Loose jumper wires on the breadboard might also cause some segments not to light up properly.

Overall, the experiment successfully achieved its purpose which was to display numbers on a 7-segment display and control them using push buttons while also demonstrating how both hardware and software work together in electronics.

Questions:

How can you interface an I2C LCD with Arduino? Explain the coding principle behind it compared to a 7-segment display and a matrix LED.

In this experiment involving the Cytron CT-UNO and a 7-segment display, the use of an I2C LCD could provide another way to visualize digital outputs while demonstrating more advanced communication methods. Unlike the 7-segment display, which requires multiple digital pins to manually control each LED segment, the I2C LCD uses only two communication lines (SDA and SCL) through serial data transfer. This greatly eases the wiring process as well as uses less pins than a 7-segment display.

In terms of programming, the difference lies in how the data is handled. For the 7-segment display, each segment's state (ON/OFF) or (HIGH/LOW) is directly controlled by digital logic outputs, making the process highly dependent on hardware wiring. In contrast, the I2C LCD uses the I2C protocol, where the microcontroller sends encoded data to the LCD's internal controller. With the help of libraries such as `LiquidCrystal_I2C`, the user can easily display characters using simple commands like `lcd.print()`. This approach abstracts the low-level control, focusing more on data display rather than bit-level manipulation.

Compared to matrix LED or 7-segment displays, the I2C LCD offers more flexibility, easier coding, and better readability for multi-character output. It also highlights how digital logic principles extend from direct pin control to serial communication systems, reinforcing the understanding of both hardware interfacing and software abstraction.

Overall, the I2C LCD interface demonstrates a practical evolution from basic digital outputs to more efficient and intelligent display systems, aligning with the experiment's goal of exploring how hardware and software integration enhances control in microcontroller-based designs.

9.

Conclusion

The experiment was successful because the Cytron CT-UNO microcontroller was able to control the 7-segment display effectively using simple push-button inputs. The display showed the correct numbers as expected, proving that the system worked as planned.

Although there was a wiring mistake the common cathode of the 7-segment display was connected to 5V instead of GND the problem was fixed through coding. By reversing the logic in the program (making LOW turn the segment ON and HIGH turn it OFF), the display still worked correctly. This shows how powerful and flexible programming can be in solving small hardware issues.

During the experiment, it was also discovered that button **debouncing** can cause problems. When a push button is pressed, it may send multiple signals instead of one, causing the display to count more than once. To reduce this problem, a short delay or debounce code can be added so that each button press is only counted once.

From this experiment, several things were learned:

- How a 7-segment display works and how each segment lights up to form numbers.
- How to use digital inputs (push buttons) and digital outputs (to control the display) in Arduino programming.
- How code logic can correct wiring mistakes and handle issues like button bouncing.

10.

In conclusion, the hypothesis that “a microcontroller can control a 7-segment display using push buttons” is supported by the results. The experiment successfully demonstrated how hardware and software can work together to create a functioning digital display system.

Recommendations

1. Correct Hardware Connection:

Always ensure the common cathode is connected to GND to avoid reversed logic, which can cause confusion during debugging.

2. Use Built-in Pull-up Resistors:

The `pinMode(button, INPUT_PULLUP)` configuration can simplify the setup and eliminate the need for external 10 k Ω resistors.

3. Add a Display Counter via Serial Monitor:

Displaying the current number on the Serial Monitor alongside the 7-segment can help verify real-time operation and debugging.

4. Improve Code Efficiency:

Using arrays or binary mappings for each digit pattern would make the code shorter and easier to modify.

5. Expand Application:

11.

Future work can integrate two 7-segment displays to show double-digit numbers or even implement a stopwatch or reaction timer using similar logic.

References

All About Circuits. (2019). *All About Circuits - Electrical Engineering & Electronics Community*. Allaboutcircuits.com. <https://www.allaboutcircuits.com/>

Arduino. (n.d.). *Arduino Docs | Arduino Documentation*. Docs.arduino.cc. <https://docs.arduino.cc/>

Cytron Technologies. (2025). *Cytron UNO - Arduino UNO Compatible*. Cytron Technologies Singapore. <https://sg.cytron.io/p-cytron-uno-arduino-uno-compatible?r=1>

International Islamic University Malaysia. (2022). *Mechatronics system integration lab manual: Week 2 – Digital logic system*.

Kuphaldt, T. R. (2015, February 18). *7-segment Display*. Allaboutcircuits.com; All About Circuits. <https://www.allaboutcircuits.com/textbook/experiments/chpt-7/7-segment-display/>

12.

Appendix A - Circuit Diagram

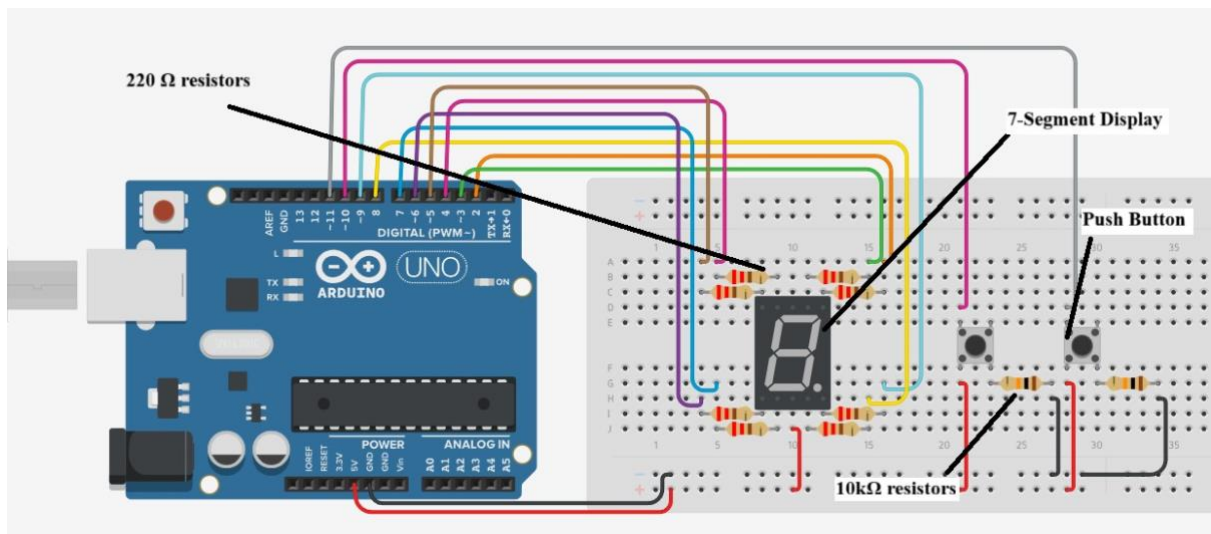


Figure A1 Shows the connection of circuit configuration by using Tinkercad

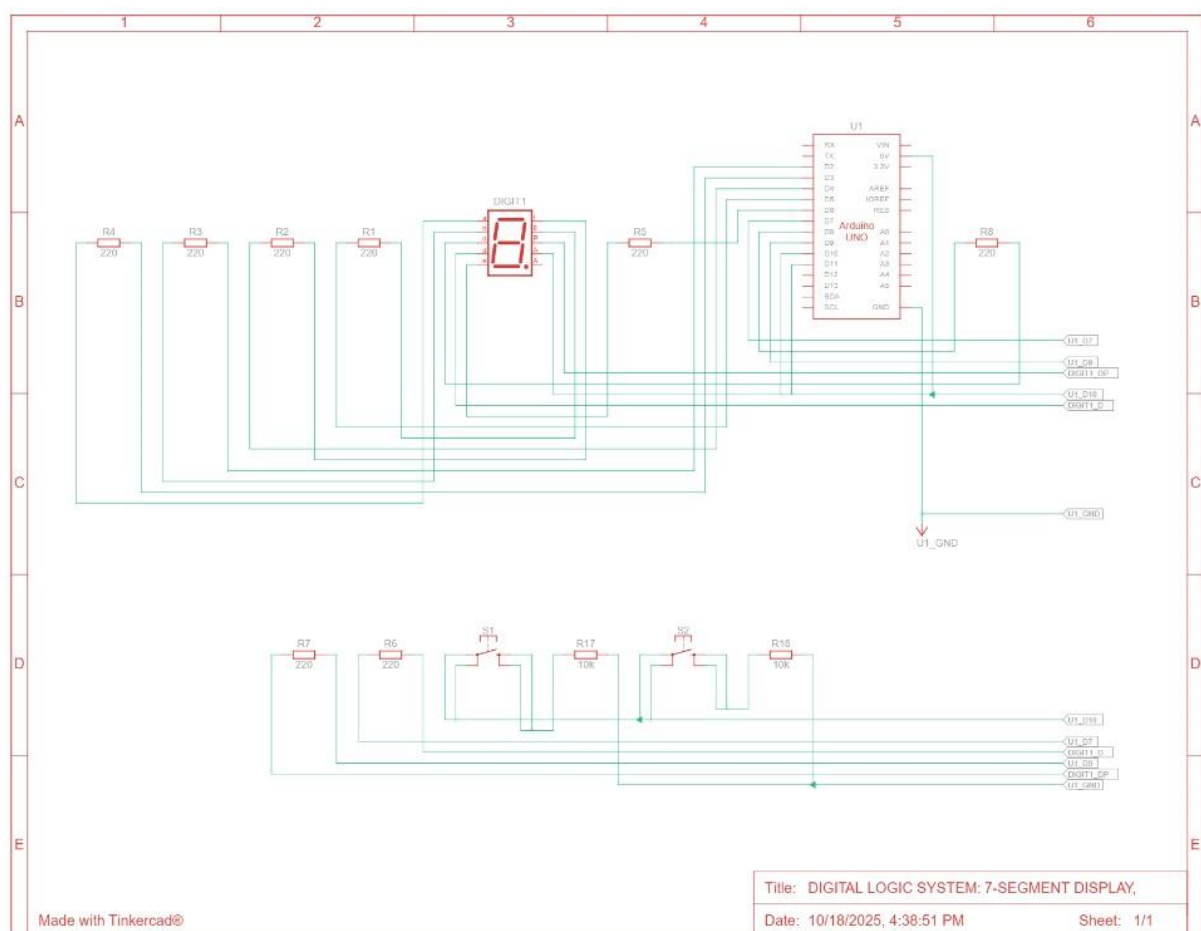


Figure A2 Shows the circuit schematic diagram by using Tinkercad

Acknowledgments

We would like to express my sincere gratitude to my lecturer and lab instructor for their continuous guidance, clear explanations, and assistance with experimental setups throughout this experiment. Their detailed instructions with the several needed files attached have helped us understand how to properly set up the Cytron CT-UNO microcontroller, connect the 7-segment display, and program the system using Arduino IDE.

I would also like to thank the teaching assistants for their patience and assistance during the lab session. Their help in identifying wiring mistakes and explaining how to correct the logic in the program was very valuable.

In addition, I truly appreciate my classmates and lab partners for their teamwork, cooperation, and ideas during the experiment. We worked together to troubleshoot connection issues, share components, and test the code to make sure everything functioned as expected.

Overall, the success of this experiment would not have been possible without the guidance and support from both the instructors and my lab partners.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specific in references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or a person.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We, therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.


Signature:

Name: MOHAMED ZAWIR FARIS BIN MOHAMED FAUZI

Matric Number: 2329493

Read /

Understand /

Agree /

Signature:

Name: MUHAMAD HARITH HAIKAL BIN ZULKIFLI

Matric Number: 2319963

Read / 

Understand /

Agree /

Signature:

Name: MUHAMMAD AIDIL AIMAN BIN WAKIL @
MUHAMAD SYUKRI

Matric Number: 2316967

Read / 

Understand /

Agree /